

URCE

通用卡牌构筑肉鸽引擎

完整设计文档 / Game Design & Technical Design Document

版本 1.0 · 2026-06-16

面向：21 点肉鸽、回合战斗爬塔、位置制献祭卡牌，以及可扩展的桌游级规则系统

版本记录

版本	日期	状态	说明
1.0	2026-06-16	首版	建立通用卡牌构筑肉鸽引擎的完整规则、内容、肉鸽流程、3D 演出和工具链设计。

阅读对象

角色	重点章节
制作人 / 创意总监	第 1 章、第 2 章、第 7 章、第 16 章、第 23 章。用于确认产品边界、玩法验证和里程碑。
系统策划 / 卡牌策划	第 4 章、第 5 章、第 8 章、第 9 章、第 10 章、第 11 章、第 14 章、第 22 章。用于设计机制、卡牌、遗物、胜利条件和关卡。
客户端 / 引擎程序	第 3 章、第 4 章、第 6 章、第 12 章、第 13 章、第 18 章、第 19 章。用于实现规则内核、状态同步、演出编排和数据管线。
TA / VFX / 动画 / UI	第 6 章、第 15 章、第 17 章。用于定义卡牌 Actor、镜头、VFX、UI 反馈和时间轴接口。
QA / 数据分析	第 20 章、第 21 章、第 24 章。用于自动化测试、回放、遥测和验收标准。

目录

1. 设计目标与范围	14. AI、敌人意图与荷官策略
2. 核心设计原则	15. 中间演出与叙事插入
3. 总体架构	16. 三类玩法落地模板
4. 规则内核	17. 数据模型与接口草案
5. 卡牌移动、销毁与生命周期	18. 3D 资产与表现绑定规范
6. 3D 演出系统	19. 存档、回放、随机与网络同步
7. 肉鸽生命周期	20. 自动化测试、平衡与遥测
8. 胜利条件与目标系统	21. 编辑器与内容生产工具
9. 回合、阶段与时机窗口	22. 开发路线图
10. 机制插件系统	23. 验收标准
11. 内容配置与 DSL	24. 风险与约束
12. 资源、经济与下注系统	25. 术语表与附录
13. 棋盘、路线与位置制	

1. 设计目标与范围

URCE 的目标是提供一套可扩展、可复现、可演出化的通用卡牌构筑肉鸽引擎。它不是单一玩法模板，而是一套规则剧场：通过对象、区域、事件、效果、触发和目标系统描述卡牌桌游逻辑，通过肉鸽框架管理整局生命周期，通过独立的 3D 演出导演把规则事实转化为华丽、可跳过、可调试的表现。

核心目标：同一套底层需要足以实现 21 点肉鸽、回合制爬塔卡牌、位置制献祭卡牌，以及混合规则的原创卡牌构筑游戏。

1.1 产品级需求

- 支持卡牌构筑、战斗/赌局/棋盘对局、奖励草案、地图节点、商店、事件、Boss、局外解锁等完整肉鸽生命周期。
- 支持新的机制以插件形式加入，包括新资源、新关键词、新区域、新时机、新效果、新 AI 评估器和新演出风格。
- 支持配置化胜利条件和失败条件，不把“击杀所有敌人”写死在战斗循环中。
- 支持卡牌移动、翻面、打出、召唤、摧毁、献祭、放逐、洗牌等 3D 演出级表现。
- 支持确定性模拟、固定种子回放、自动化平衡测试、战斗日志和可追踪 Debug。

1.2 非目标

- 不把玩法绑定到某个具体游戏的数值、UI 或关卡结构。
- 不要求规则内核依赖特定商业引擎；Unity、Unreal 或自研引擎都应可接入。
- 不让表现动画直接修改规则状态。动画可以延迟、加速或跳过，但规则结果必须确定。
- 不把所有规则都写成硬编码；常规卡牌、遗物、敌人与事件应以数据配置为主。

2. 核心设计原则

2.1 规则只产生事实，演出负责表现事实

规则层只记录“发生了什么”：卡牌从哪个区域移动到哪个区域、对象受到多少伤害、资源如何变化、哪些触发进入队列。演出层只负责“如何表现”：镜头、动画、音效、粒子、UI 节奏和慢动作。二者通过事件日志和表现意图连接。

规则事实与表现意图分离

```
Command: PlayCard(card_12, target_enemy_1)
Rule Events:
  ResourceChanged(energy -1)
  CardMoved(hand -> play_stack)
  DamageDealt(amount=6)
  CardMoved(play_stack -> discard)
Presentation Intents:
  CardThrow(card_12 -> enemy_1)
  SlashImpact(enemy_1)
  DamageNumber(6)
  CardFlyToDiscard(card_12)
```

2.2 数据驱动优先，代码扩展兜底

普通卡牌、遗物、奖励、敌人意图、事件节点和胜负条件应由声明式数据描述；只有 Boss、多阶段谜题、特殊叙事或需要复杂搜索的机制才进入沙盒脚本。所有脚本都必须通过受控 Context API 修改状态。

2.3 可复现优先

任何影响规则的随机数必须来自可记录、可回放的规则 RNG 流。表现粒子的随机、镜头轻微漂移、音效变体等必须使用独立表现 RNG，不能污染规则复现。

2.4 桌游级术语与时机模型

引擎采用经典卡牌/桌游设计中的对象、区域、费用、效果、触发、替代、防止、持续效果、层、优先权、状态检查动作、最后已知信息等概念。这样能承载复杂机制，而不是在内容规模扩大后被特殊规则拖垮。

3. 总体架构

URCE 由六大层组成：规则内核、内容系统、肉鸽框架、表现编排、编辑器工具和模拟调试。各层之间通过清晰接口通信，避免内容、规则与表现互相耦合。

层级	职责	主要产物
Rules Core / 规则内核	确定性模拟游戏状态、命令、事件、效果、触发、目标和随机。	GameState、EventLog、EffectResolver、TriggerQueue、ObjectiveGraph。
Content System / 内容系统	定义卡牌、遗物、敌人、关键词、机制、奖励、事件和关卡参数。	CardDefinition、RelicDefinition、MechanicPlugin、RewardPool。
Run Framework / 肉鸽框架	管理新局、地图、节点、奖励、商店、休息、Boss、失败和局外成长。	RunState、MapGraph、NodeResolver、RewardDraft。
Presentation Orchestrator / 表现编排	将规则事件转成 3D 卡牌、镜头、VFX、音效和 UI 节奏。	PresentationIntent、BeatTimeline、CameraDirector、CardActor。
Authoring Tools / 编辑器工具	支撑策划和美术生产内容、验证规则和绑定资产。	卡牌编辑器、遭遇编辑器、演出时间轴、规则调试器。
Simulation & Debug / 模拟调试	进行自动对局、平衡分析、回放、差异检测和异常断言。	Replay、Golden Tests、Telemetry、Balance Dashboard。

总体数据流

Input / AI Command	
-> Rules Core	
-> Event Ledger	
-> Objective Check	
-> Presentation Intent Builder	
-> Beat Scheduler	
-> 3D Card Actors / Camera / VFX / Audio / UI	
Content Data -> Rules Core	
Run State -> Rules Core	
Tools -> Content Data + Validation + Replay	

3.1 推荐模块命名

URCE.Core	GameState, ObjectModel, ZoneSystem, EventSystem, RNG
URCE.Cards	CardDefinitions, Costs, Targeting, Movement, Keywords
URCE.Roguelike	RunState, MapGeneration, Nodes, Rewards, Shops
URCE.Board	Slots, Lanes, Grids, CombatResolution
URCE.Blackjack	Shoe, HandValue, DealerPolicy, Betting, Payout
URCE.Presentation	Actors, Anchors, Timelines, Camera, VFX, Audio, UI
URCE.Authoring	DSL, Validators, Editors, Localization, AssetBinding
URCE.Simulation	Bots, Replays, Tests, Telemetry, Balance Dashboard

4. 规则内核

4.1 GameState

GameState 是规则层唯一可信事实源。它不保存动画状态，不保存 UI 临时选择，不保存粒子或镜头状态。GameState 必须可序列化、可克隆、可比较，并能从 CommandLog + Seed 复现。

```
GameState {
  gameId: string
  rulesetId: string
  phase: PhaseId
  activePlayer: PlayerId
  players: PlayerState[]
  objects: Map<ObjectId, GameObject>
  zones: Map<ZoneId, Zone>
  resources: ResourceState
  objectives: ObjectiveGraph
  triggerQueue: Trigger[]
  replacementEffects: ReplacementEffect[]
  eventLog: Event[]
  rngState: RngState
}
```

4.2 GameObject

一切可被规则引用的实体都是 GameObject：卡牌、衍生物、敌人、遗物、状态、敌人意图、棋盘格规则、目标、剧情旗标、场地光环等。

字段	含义
id	局内唯一对象 ID。
definitionId	局外定义模板 ID。
owner	原始所有者。
controller	当前操控者。
type	Card、Token、Relic、Enemy、Status、Intent、Objective、RuleAura 等。
zone	所在区域引用。
visibility	公开、隐藏、仅所有者可见、指定玩家可见。
characteristics	攻击、生命、费用、点数、花色、稀有度等可被修正的特征。
counters	毒、燃烧、充能、护甲、+1/+1 等指示物。
attachments	贴附物，如印记、装备、诅咒、升级、符文。
stateFlags	临时状态，如 exhausted、marked、asleep、destroyed。

4.3 CardDefinition 与 CardInstance

必须区分卡牌定义和卡牌实例。定义是局外模板，实例是局内对象。这样才能支持升级、复制、永久改造、临时生成、剧情卡、贴附印记、卡面被污染等效果。

```
CardDefinition {
  id: "card.strike"
  name: "Strike"
  baseTypes: ["Attack"]
  cost: CostExpression
}
```

```

text: LocalizedText
keywords: KeywordRef[]
effects: EffectScript[]
presentation: PresentationProfileRef
rarity: Common | Uncommon | Rare | Special
pools: ["starter", "warrior"]
}

CardInstance {
  objectId: "card_1034"
  definitionId: "card.strike"
  owner: "player"
  upgrades: UpgradeState[]
  modifications: RuntimeModifier[]
  currentCost: number
  createdBy: ObjectId?
}

```

4.4 Zone 系统

Zone 是所有卡牌移动的基础。不要把“牌堆、手牌、弃牌堆、墓地、放逐、商店”写成硬编码列表，而应统一为可配置容器。

```

Zone {
  id: "player.hand"
  owner: PlayerId?
  kind: OrderedStack | OrderedList | UnorderedBag | Grid | LaneBoard | Market | Timeline
  visibility: Public | Hidden | OwnerOnly
  capacity: number | Infinite
  ordering: TopFirst | BottomFirst | Manual | Randomized
  topology?: GridTopology | LaneTopology
}

```

标准区域	用途
Draw Pile / Library / Shoe	抽牌堆、牌库或牌靴。
Hand	手牌。
Play Stack / Action Queue	正在打出或等待结算的牌与效果。
Board / Battlefield / Table	场上区域。
Lane / Slot	位置制卡牌的路线或格子。
Discard Pile	弃牌堆。
Graveyard	墓地，适合实体死亡后进入。
Exhaust / Exile / Void	移出本场战斗、放逐或永久移除区。
Market / Shop	商店展示区。
Reward Pool	奖励候选池。
Encounter Queue	敌方行动、未来卡牌或叙事事件队列。

4.5 Command 管线

玩家输入、AI 行动、脚本行动都必须走统一 Command 管线。任何跳过合法性、费用、事件和触发的特殊入口都会破坏回放和扩展性。

```

Declare Command
-> Validate Legality
-> Choose Modes / Targets / Values
-> Pay Costs

```

- > Resolve Effects
- > Emit Events
- > Check State-Based Actions
- > Queue Triggers
- > Resolve Trigger Queue
- > Emit Presentation Intents
- > Commit State Snapshot

4.6 Event 与 Event Ledger

事件是规则历史中的事实。所有规则事件都应带来源、原因、可见性和规则时间戳。表现层、日志、回放、触发系统和遥测都依赖 Event Ledger。

```
Event {
  id: EventId
  type: "CardMoved" | "CardPlayed" | "DamageDealt" | "ResourceChanged" | ...
  source: ObjectId?
  cause: CauseRef
  payload: any
  timestamp: RuleTime
  visibility: Visibility
}
```

事件类型	说明
BeforeCardPlayed / AfterCardPlayed	打出卡牌前后。
BeforeMove / AfterMove	对象移动前后。
BeforeDestroy / AfterDestroy	销毁、死亡、献祭前后。
BeforeDamage / AfterDamage	伤害修正、伤害应用、受伤触发。
ResourceChanged	能量、金币、血、骨头、赌注、怀疑值等变化。
CounterAdded / CounterRemoved	指示物变化。
PhaseEntered / PhaseExited	阶段切换。
ObjectiveChecked	胜负条件检查。
RandomRolled	影响规则的随机结果。
PresentationBeatRequested	规则请求表现节拍。

4.7 Effect 系统

Effect 是改变 GameState 的原子操作。卡牌脚本不应直接修改对象字段，而应组合标准 Effect，以便验证、重放、统计和表现绑定。

Effect 类别	典型效果
对象与区域	MoveObject、DrawCards、Reveal、Shuffle、Search、CreateObject、Copy、Transform、Attach、Detach。
移除与死亡	Destroy、Sacrifice、Discard、Exhaust、Exile、RemoveFromGame。
战斗数值	DealDamage、Heal、GainBlock、GainShield、ModifyCharacteristic。
资源经济	GainResource、SpendResource、SetBet、Payout、AddCounter、RemoveCounter。
控制流	ChooseOne、Repeat、Conditional、ScheduleDelayedEffect、EndTurn、WinObjective、LoseObjective。
21 点扩展	DealFromShoe、Hit、Stand、DoubleDown、SplitHand、

	BustCheck、CompareHandTotal。
位置制扩展	OccupySlot、VacateSlot、AttackLane、BlockLane、SummonToSlot、AdvanceQueue。

4.8 Trigger、Replacement 与 Prevention

Trigger 监听已发生或将发生的事件；Replacement 可以把将要发生的事替换成另一件事；Prevention 用于防止伤害、移动、死亡或资源损失。这三类机制是承载高级卡牌交互的基础。

```

Trigger {
  listen: EventPattern
  condition: Predicate
  timing: Before | After | Replacement | Prevention | Interrupt
  priority: number
  effect: EffectScript
  onceLimit?: OncePerTurn | OncePerCombat | OncePerRun
}

```

4.9 持续效果与层

持续效果不能按触发顺序简单叠加，否则复杂数值会不可控。引擎应采用层系统，按固定层级计算最终特征。

层	内容
1 基础定义层	读取卡牌、敌人或对象的原始定义。
2 复制/变形层	复制另一对象或变形成另一对象。
3 控制权/所属权层	改变 controller 或 owner 的规则处理。
4 类型/标签层	改变 Attack、Skill、Creature、Cheat、Curse 等类型与标签。
5 数值设定层	攻击力变为 1、费用变为 0。
6 数值加减层	伤害+2、费用-1、护甲+5。
7 倍率层	下一张攻击牌伤害翻倍。
8 限制层	不能攻击、不能被指定、费用不能低于 0。
9 展示层	名称、描述、视觉标签、卡面特效。

5. 卡牌移动、销毁与生命周期

5.1 MoveObject 是第一等规则操作

卡牌移动不是动画事件，而是规则事件。抽牌、打出、弃牌、洗牌、死亡、放逐、召唤、回手、从商店加入牌组都应统一建模为 MoveObject 或其组合。

```

MoveObject {
  object: ObjectId
  from: ZoneRef
  to: ZoneRef
  position?: ZonePosition
  face?: FaceUp | FaceDown | Preserve
  visibility?: VisibilityChange
  cause: Draw | Play | Discard | Destroy | Sacrifice | Exhaust | Shuffle | Scripted
  atomicGroup?: GroupId
}

```


5.2 MoveObject 结算流程

```
BeforeMove
-> Apply Replacement Effects
-> Validate destination capacity and visibility
-> LeaveZone
-> EnterZone
-> AfterMove
-> ZoneStateCheck
-> Queue Triggers
-> Build PresentationIntent
```

5.3 Destroy 不是 Delete

销毁必须是流程，而不是直接删除对象。对象被销毁时可能触发亡语、替代为放逐、留下残片、进入墓地、转化为幽灵、触发叙事，或者被不死效果拦截。

```
Destroy(object)
-> BeforeDestroy
-> Replacement / Prevention check
-> MarkAsDestroyed
-> LeavePlay
-> Death / Sacrifice / Destroyed triggers
-> MoveToDestination(Graveyard | Discard | Exhaust | Void)
-> AfterDestroy
-> Cleanup references
```

5.4 最后已知信息

当对象离开场上或被摧毁后，后续触发仍可能需要它离场前的信息，如攻击力、控制者、是否被献祭、所在格子、卡面印记等。引擎应在关键事件中记录 Last Known Information。

场景	需要记录的信息
死亡触发	死亡前 controller、owner、类型、攻击、生命、关键词、所在 Slot。
伤害来源离场	来源对象的定义、伤害类型、控制者、标签。
卡牌被打出后进入弃牌堆	打出时费用、目标、是否被复制、是否受某遗物影响。
献祭	献祭原因、被献祭对象数量、献祭前的价值。

6. 3D 演出系统

演出系统的目标是达到演出级别的实体卡牌表现，同时不破坏规则确定性。它以 Event Ledger 为输入，输出镜头、动画、VFX、音效和 UI 反馈。

6.1 PresentationIntent

```
PresentationIntent {
  eventId: EventId
  type: "CardMove" | "CardDestroy" | "Damage" | "Summon" | "Transform" | "Reveal"
  subject: ObjectId
  fromAnchor: PresentationAnchor
  toAnchor: PresentationAnchor
  style: "Elegant" | "Violent" | "Arcane" | "Horror" | "Casino" | "Mechanical"
  intensity: 0.0..1.0
  durationPolicy: Instant | Fast | Normal | Cinematic | SkippableCinematic
}
```

```
syncGroup?: string
}
```

6.2 CardActor

每张可见卡牌在表现层对应一个 CardActor。CardActor 绑定 GameObject ID，但只保存表现状态，不保存规则数值。

```
CardActor {
  objectId: ObjectId
  mesh: CardMesh
  frontMaterial: Material
  backMaterial: Material
  borderMaterial: Material
  foilLayer: ShaderLayer
  decalLayers: Decal[]
  statusVfxSockets: Socket[]
  animationState: CardAnimState
  anchorBinding: ZoneAnchorRef
}
```

表现能力	用途
Flip	翻面、揭示、荷官翻牌。
Fan	手牌扇形排列。
Hover Inspect	悬停放大审视。
Throw / Cast	打出卡牌时飞向目标或桌面中心。
Slam	重击落牌、Boss 召唤。
Burn / Dissolve	销毁、放逐、诅咒烧毁。
Shatter	碎裂、破坏、封印解除。
Ink Bleed	墨迹扩散，适合恐怖或污染机制。
Foil Shine	稀有牌、传奇牌、胜利奖励。
Sigil Carving	印记刻入、融合、永久改造。
Casino Flick	赌场式弹牌、发牌、切牌。
Soul Absorb	灵魂、骨头、金币、诅咒流入卡面。

6.3 Presentation Director

Presentation Director 是演出导演。它不直接问卡牌效果发生了什么，而是读取事件日志和表现意图，合并、排序、升格、调度演出节拍。

```
Rule Event Ledger
-> Presentation Intent Builder
-> Beat Scheduler
-> Camera Director
-> VFX Director
-> Audio Director
-> UI Director
-> Timeline Playback
```

6.4 Beat 时间轴模型

```
PresentationBeat {
  id: BeatId
  startCondition: AfterPrevious | WithPrevious | OnEvent | Delay
  duration: Seconds
}
```

```
actorRefs: ActorRef[]
actions: PresentationAction[]
canSkip: boolean
gameplayBlocking: boolean
}
```

Beat 动作	说明
FocusCamera	镜头聚焦到卡牌、目标、牌堆、Boss 或筹码。
MoveActor	沿贝塞尔曲线、抛物线、螺旋路径移动。
FlipCard	翻面并同步揭示信息。
MaterialDissolve	溶解、烧毁、消散。
SpawnVFX	粒子、轨迹、冲击波、墨迹、灵魂。
ShakeCamera	震动和冲击反馈。
PlaySound	纸牌摩擦、筹码、刀光、火焰、低语。
ShowText	伤害数字、Bust、Perfect、Combo、Critical。
WaitForImpact	等待动画到达命中帧后再播放后续表现。

6.5 卡牌移动演出模板

```
CardMove_Normal:
1. Lift card from source anchor
2. Add rim light and slight scale punch
3. Move along curved path to destination anchor
4. Rotate or flip if visibility changes
5. Snap to zone layout
6. Play paper landing sound

CardDestroy_Epic:
1. Focus camera and lower ambient light
2. Edge burn or crack material starts
3. Card bends, shakes, emits ash particles
4. Object moves toward Graveyard/Void anchor
5. Dissolve mesh and leave residual particles
6. Continue death trigger presentation
```

6.6 表现优先级

等级	使用场景	策略
Instant	大量小事件、自动模拟、快速模式。	只更新 UI 或做极短动画。
Fast	普通抽牌、弃牌、资源变化。	短飞行、轻音效、无镜头切换。
Normal	打牌、攻击、召唤、触发遗物。	标准卡牌飞行、VFX、命中反馈。
Cinematic	稀有牌、Boss 阶段、关键死亡、重大剧情。	镜头、慢动作、灯光、特效。
SkippableCinematic	首次出现的中间演出、结局、Boss 变身。	可跳过，跳过后规则状态保持一致。

7. 肉鸽生命周期

一套完整卡牌构筑肉鸽引擎必须覆盖从局外档案到单局结束后的统计与解锁，而不是只覆盖战斗。

```
Profile / Meta
-> New Run
-> Run Seed Created
```

- > Character / Ruleset / Starter Deck Selected
- > World / Map Generated
- > Act Loop
 - > Choose Node
 - > Enter Node
 - > Encounter / Event / Shop / Rest / Treasure / Boss
 - > Resolve Node
 - > Reward Draft
 - > Deck Modification
 - > Resource Settlement
 - > Map Progression
- > Run Victory / Run Defeat
- > Unlocks / Statistics / Replay / Codex Update

7.1 RunState

```
RunState {
  seed: string
  rulesetId: string
  playerProfile: ProfileRef
  deck: CardInstance[]
  relics: ObjectId[]
  runResources: ResourceMap
  map: MapGraph
  currentNode: NodeId
  act: number
  difficulty: DifficultyState
  flags: NarrativeFlag[]
  history: RunEvent[]
}
```

7.2 地图节点类型

节点	说明
Combat	普通战斗、普通赌局或普通棋盘对局。
Elite	高风险高奖励遭遇。
Boss	Act Boss 或最终 Boss。
Event	叙事事件、选择事件、风险事件。
Shop	商店、黑市、荷官摊位、神秘商人。
Rest	休息、升级、治疗、删牌、净化。
Treasure	宝箱、遗物、特殊牌。
Gamble	下注、翻倍、轮盘、21 点桌。
Sacrifice / Forge	献祭、融合、刻印、锻造。
Puzzle	谜题、密室、卡牌排列挑战。
Narrative Interlude	不可战斗的中间演出或剧情插入。
Rule Mutation	修改本局规则、加入诅咒或祝福。

7.3 Reward Draft

奖励不应写死为“三选一卡”。RewardOffer 可以代表卡牌、遗物、金币、升级、删牌、融合、诅咒、规则修改或剧情旗标。

```
RewardOffer {
  id: RewardId
  category: Card | Relic | Currency | Upgrade | RemoveCard | TransformCard | Heal | Curse | RuleModifier
```

```

choices: RewardChoice[]
pickCount: number
skipAllowed: boolean
rarityWeights: WeightedTable
tags: Tag[]
}

```

8. 胜利条件与目标系统

胜负条件必须配置化。不同模式可能是击杀敌人、筹码达到阈值、天平达到一侧、存活指定回合、解开谜题、完成剧情目标或击败最终荷官。

```

Objective {
  id: "defeat_all_enemies"
  scope: Encounter | Act | Run | Meta
  predicate: PredicateScript
  result: Victory | Defeat | Draw | Branch
  priority: number
  tiebreaker?: TiebreakerScript
  presentation?: PresentationProfileRef
}

```

8.1 常规战斗胜负

```

objectives:
- id: defeat_all_enemies
  scope: Encounter
  predicate: "enemies.all(e => e.hp <= 0)"
  result: Victory
- id: player_dead
  scope: Encounter
  predicate: "player.hp <= 0"
  result: Defeat
  priority: 100

```

8.2 21 点胜负

```

objectives:
- id: player_bust
  predicate: "player.handTotal > 21"
  result: Defeat
- id: dealer_bust
  predicate: "dealer.handTotal > 21"
  result: Victory
- id: compare_totals
  predicate: "phase == Compare"
  result: "Compare(player.handTotal, dealer.handTotal)"

```

8.3 天平与棋盘胜负

```

objectives:
- id: scale_reached_enemy_side
  predicate: "scale.value >= 5"
  result: Victory
- id: scale_reached_player_side
  predicate: "scale.value <= -5"

```

```
result: Defeat
```

8.4 目标优先级与平局

多个目标可能在同一状态检查中同时满足，例如玩家和 Boss 同时死亡。ObjectiveGraph 必须支持 priority 与 tiebreaker。默认策略建议为：剧情强制目标优先，失败目标高于普通胜利目标，特殊遗物或合同可覆盖默认失败。

9. 回合、阶段与时机窗口

URCE 不假设所有游戏都是“玩家回合 -> 敌人回合”。阶段机应由 Ruleset 配置，并为每个阶段提供可执行命令、自动效果和触发窗口。

```
PhaseMachine {  
  phases: Phase[]  
  transitions: Transition[]  
  hooks: PhaseHook[]  
}
```

模式	阶段示例
回合战斗爬塔	EncounterStart -> PlayerTurnStart -> Draw -> PlayerAction -> PlayerTurnEnd -> EnemyIntentResolve -> EnemyTurnEnd -> CheckVictory。
21 点赌局	RoundStart -> Ante -> InitialDeal -> PlayerDecision -> DealerReveal -> DealerPolicy -> Compare -> Payout -> RoundEnd。
献祭棋盘	RoundStart -> DrawChoice -> MainAction -> SacrificeAndSummon -> LaneCombat -> OpponentAdvance -> Cleanup。
传统 TCG	Start -> Upkeep -> Draw -> Main -> Combat -> Main2 -> End -> Cleanup。

9.1 时机窗口规范

- 每个阶段进入时发出 PhaseEntered，退出时发出 PhaseExited。
- 每个命令执行前后都应有 BeforeCommand 与 AfterCommand 或具体事件。
- 触发队列必须有稳定排序：优先级、控制者、创建时间、对象 ID。
- 替代和防止效果只在 Before 事件窗口生效，避免事后撤销造成状态不一致。
- 状态检查动作在每个效果结算后、触发入队前执行。

10. 机制插件系统

新机制必须能以插件包加入，而不是修改核心规则。插件可以注册资源、关键词、区域、效果、触发模板、UI 面板、演出配置和 AI 评估器。

```
MechanicPlugin {  
  id: "blood_sacrifice"  
  resources: ResourceDefinition[]  
  keywords: KeywordDefinition[]  
  zones: ZoneDefinition[]  
  effects: EffectDefinition[]  
  triggers: TriggerTemplate[]  
}
```

```

    uiPanels: UIPanelDefinition[]
    presentationProfiles: PresentationProfile[]
    aiEvaluators: AIEvaluator[]
    balanceMetrics: MetricDefinition[]
  }

```

10.1 机制上线清单

维度	必须回答的问题
Rule	机制如何结算？是否需要新效果、新触发、新区域或新状态检查动作？
Content	哪些卡、敌人、遗物、事件和奖励使用该机制？
UX	玩家如何理解、选择、预览和撤销？是否需要教程提示？
AI	敌人或自动模拟如何评估该机制？
Presentation	它在 3D 中如何表现？需要哪些动画、VFX、音效、镜头？
Balance	需要记录哪些指标？是否有爆炸性组合风险？

10.2 献祭机制插件示例

```

mechanic: sacrifice
costs:
  - id: SacrificeCreatures
    params:
      count: int
      predicate: "object.type == Creature && object.controller == payer"
effects:
  - id: Sacrifice
    result:
      - EmitEvent: BeforeSacrifice
      - Destroy:
          cause: Sacrifice
      - EmitEvent: AfterSacrifice
presentation:
  style: Horror
  beats:
    - focus_subject
    - red_thread_bind
    - card_scream
    - dissolve_to_blood
    - resource_orb_gain

```

11. 内容配置与 DSL

内容配置应覆盖卡牌、遗物、敌人、事件、奖励和演出绑定。建议采用声明式 YAML/JSON，同时提供可视化编辑器。

11.1 基础攻击牌

```

id: card.strike
name: Strike
types: [Attack]
cost:
  energy: 1
targets:
  - id: enemy
    zone: enemy.board

```

```

    predicate: "object.type == Enemy && object.hp > 0"
effects:
  - DealDamage:
      target: selected.enemy
      amount:
        base: 6
        scaling:
          - stat: strength
presentation:
  playStyle: Slash
  cardMove: ThrowToTarget
  impactVfx: SmallCut

```

11.2 抽牌弃牌牌

```

id: card.quick_gambit
name: Quick Gambit
types: [Skill, Gamble]
cost:
  energy: 1
effects:
  - DrawCards:
      player: owner
      amount: 2
  - Discard:
      player: owner
      amount: 1
      selection: PlayerChoice
presentation:
  playStyle: Elegant
  beats:
    - card_fan_expand
    - two_cards_fly_from_deck
    - discard_card_spins_to_pile

```

11.3 21 点作弊牌

```

id: card.loaded_ace
name: Loaded Ace
types: [Cheat, CardModifier]
cost:
  coins: 3
timing:
  playableDuring: PlayerDecision
effects:
  - ChooseCard:
      zone: player.hand
      predicate: "card.rank == 'A'"
      bind: chosen
  - ModifyCharacteristic:
      object: chosen
      key: blackjackValue
      value: 11
      duration: UntilRoundEnd
  - AddCounter:
      object: player
      counter: suspicion
      amount: 1
presentation:
  playStyle: CasinoCheat

```



```
beats:
  - camera_close_on_sleeve
  - hidden_card_swap
  - golden_ace_flash
  - suspicion_meter_tick
```

11.4 生物与印记牌

```
id: card.wolf
name: Wolf
types: [Creature]
cost:
  sacrifice:
    count: 2
    predicate: "object.type == Creature && object.controller == owner"
characteristics:
  attack: 3
  health: 2
keywords:
  - lane_attack
effects:
  - SummonToSlot:
    slot: selected.own_empty_slot
presentation:
  playStyle: HorrorSummon
beats:
  - sacrifices_bleed_into_table
  - card_slam_to_slot
  - claw_marks_appear
```

11.5 遗物 / Artifact

```
id: relic.first_blood_coin
name: First Blood Coin
type: Relic
triggers:
  - listen: DamageDealt
    condition: "event.source.controller == owner && event.target.wasAtFullHealth"
    once: PerEncounter
    effect:
      GainResource:
        player: owner
        resource: coins
        amount: 5
presentation:
  triggerStyle: CoinBurst
```

11.6 数据验证规则

- 所有 definitionId 必须唯一。
- Effect 引用的 target、resource、zone、keyword 必须存在。
- 目标选择器必须能在 UI 中表达，不能只在脚本中隐式选择。
- 费用、收益、触发次数、持续时间必须有范围校验。
- 任何可进入奖励池的内容都必须声明 rarity、pool、unlockCondition。
- 所有表现 profile 必须有安全 fallback，避免缺资产导致规则卡死。

12. 资源、经济与下注系统

引擎不应只支持 Energy。资源可按回合、遭遇、整局或局外档案维度存在，并可设置刷新策略、上限、可见性和表现方式。

```
ResourceDefinition {
  id: "energy"
  scope: Turn | Encounter | Run | Meta
  min: number
  max: number | Infinite
  refreshPolicy: PhaseBased | Manual | Never | Custom
  visibility: Public | Hidden
}
```

资源	使用场景
Energy / Mana / Action Points	回合制打牌资源。
Health / Life / Scale	生命、天平、赌桌胜负值。
Block / Armor / Shield	临时防御。
Gold / Coins / Soul Coins	商店、赌博、剧情交易。
Bet / Wager	下注金额、翻倍、保险。
Blood / Bones / Sacrifice Count	献祭与死亡收益。
Heat / Suspicion	作弊风险、追捕等级。
Corruption / Curse	负面成长、代价型奖励。
Combo / Momentum	连击、节奏、手牌链式奖励。
Time / Countdown	Boss 倒计时、事件倒计时。

12.1 经济来源与水槽

类型	例子	设计目的
来源	战斗胜利、赌局获胜、遗物触发、事件奖励、Boss 宝箱。	提供成长和正反馈。
水槽	商店购买、删牌、升级、治疗、下注、解咒、开门费用。	控制经济膨胀并制造选择。
风险来源	接受诅咒换金币、作弊换收益、献祭永牌。	制造肉鸽张力。
保底机制	失败补偿、稀有度 pity、关键资源底线。	降低极端随机导致的无趣失败。

13. 棋盘、路线与位置制

为了支持位置制卡牌、路线攻击、献祭召唤、敌方队列推进和格子地形，Board 不能只是“场上列表”。

```
Board {
  id: "main_board"
  topology: LaneTopology | GridTopology | RingTopology | FreeformTopology
  slots: Slot[]
}

Slot {
  id: "player_lane_1"
  ownerSide: Player
  coordinates: Vec2 | Vec3
  opposite?: SlotId
  neighbors: SlotId[]
  capacity: 1
}
```

```
terrainTags: Tag[]
}
```

位置制机制	说明
Opposing Slot	对位格，用于结算正面攻击。
Lane Attack	沿路线攻击，对位有单位则打单位，无单位则打玩家或天平。
Blocker	阻挡者，保护后排或玩家。
Frontline / Backline	前后排。
Adjacency	相邻格影响，如相邻单位+1 攻击。
Range	射程与可指定范围。
Advance Queue	敌方卡牌从队列推进到棋盘。
Occupy / Vacate	占据与离开格子事件。
Terrain Modifier	地形、陷阱、祭坛、赌场桌格等。

13.1 Lane Combat 流程

```
For each lane in attackOrder:
  attacker = activeSide.slot[lane].object
  blocker  = opposingSide.slot[lane].object
  if attacker is empty: continue
  if blocker exists:
    DealDamage(attacker.attack -> blocker)
  else:
    DealDirectDamage(attacker.attack -> opposingScaleOrPlayer)
  CheckStateBasedActions()
  QueueDeathAndDamageTriggers()
```

14. AI、敌人意图与荷官策略

AI 通过 Agent Policy 控制。不同模式可以使用脚本序列、效用 AI、行为树、蒙特卡洛搜索、荷官固定策略或叙事 AI。

```
Agent {
  id: AgentId
  controlledSide: SideId
  policy: ScriptedPattern | UtilityAI | BehaviorTree | MCTS | DealerPolicy | PuzzleSolver
  knowledgeModel: Perfect | PublicOnly | Cheating | Narrative
}
```

AI 类型	适用场景
Scripted Pattern	固定敌人行动序列、教学战斗。
Intent Table	提前展示敌人下回合意图。
Dealer Policy	21 点荷官按规则摸牌或停牌。
Utility AI	根据伤害、风险、资源收益评估行动。
Behavior Tree	Boss 多阶段、多条件行为。
MCTS / Search	复杂对手或自动平衡模拟。
Narrative AI	会说话、作弊、插入剧情或打破规则的对手。

14.1 敌人意图是对象

```
id: intent.enemy_heavy_attack
type: Intent
```

```
visibility: Public
effects:
  - DealDamage:
      target: player
      amount: 18
presentation:
  telegraph: RedSword
  intensity: High
```

14.2 AI 评估接口

```
AIEvaluator {
  scoreCommand(command, simulatedState): number
  explain(command, simulatedState): DebugReason[]
}

Common features:
  expectedDamage
  survivalProbability
  resourceDelta
  cardAdvantage
  boardControl
  bustRisk
  suspicionRisk
  sacrificeValue
```

15. 中间演出与叙事插入

中间演出用于 Boss 阶段转换、剧情打断、首次见到机制、作弊暴露、特殊卡牌觉醒、谜题解锁等。它可以阻塞玩法，也可以作为后台演出。

```
Interlude {
  id: "boss_phase_2_reveal"
  trigger: EventPattern | ObjectivePredicate | NodeEnter
  blocking: true | false
  timeline: TimelineAssetRef
  choices?: Choice[]
  effectsAfter?: EffectScript[]
}
```

15.1 触发位置

- 进入地图节点前或节点结算后。
- 战斗开始、首次抽到某张牌、首次作弊、首次献祭。
- Boss 血量或天平达到阈值。
- 某个剧情旗标被设置。
- 玩家死亡但被遗物、合同或剧情规则复活。
- 胜利结算前、奖励草案前或进入下一 Act 前。

15.2 中间演出示例

```
id: interlude.dealer_reveals_true_face
trigger:
  listen: ResourceChanged
  condition: "resource == soulCoins && player.soulCoins >= 50"
```

```
blocking: true
timeline: "cinematics/dealer_true_face.timeline"
effectsAfter:
  - AddRunFlag: dealer_true_face_seen
  - AddRelic:
      id: relic.cursed_chip
```

16. 三类玩法落地模板

16.1 21 点肉鸽规则集模板

模块	配置
Zones	shoe、player.hand、dealer.hand、discard、cheat_stash、market。
Resources	soulCoins、bet、suspicion、luck、debt。
Phases	Ante、InitialDeal、PlayerDecision、DealerReveal、DealerPolicy、Compare、Payout。
Commands	Hit、Stand、DoubleDown、Split、Cheat、BuyInsurance、Surrender。
Objectives	爆牌失败、荷官爆牌胜利、Compare 阶段比较点数、赚够筹码胜利。
Presentation	牌靴发牌、筹码物理、荷官手部动画、暗中换牌、爆牌火焰、金币流入 UI。

```
Hit:
  effects:
    - DealFromShoe:
        from: shoe
        to: player.hand
        reveal: true
    - BustCheck:
        hand: player.hand

DealerPolicy:
  while dealer.handTotal < 17:
    DealFromShoe(shoe -> dealer.hand)
  transition: Compare
```

16.2 回合制爬塔规则集模板

模块	配置
Zones	drawPile、hand、discardPile、exhaustPile、enemyIntentZone。
Resources	hp、energy、block、gold、potionSlots。
Objects	cards、relics、enemies、powers、statuses、potions。
Phases	EncounterStart、PlayerTurnStart、Draw、PlayerAction、EnemyIntentResolve、Cleanup。
Objectives	所有敌人死亡胜利、玩家 HP 归零失败。
Presentation	卡牌飞向目标、敌人意图悬浮、遗物触发光效、抽弃牌堆实体化。

16.3 位置制献祭规则集模板

模块	配置
Zones	player.hand、player.boardSlots、opponent.boardSlots、opponent.queue、sacrificeZone、graveyard、items。
Resources	scale、blood、bones、teeth、candles。
Board	4 条或 N 条路线，每条路线有玩家格、敌人格、对位关系。
Commands	DrawChoice、PlayCreature、Sacrifice、UseItem、EndTurn。
Objectives	天平达到敌方侧胜利，达到玩家侧失败；Boss 可覆盖为多阶段目标。
Presentation	昏暗木桌、蜡烛、血迹、卡牌刻印、献祭渗血、路线攻击实体化。

17. 数据模型与接口草案

17.1 Ruleset

```
RulesetDefinition {
  id: string
  displayName: LocalizedText
  plugins: MechanicPluginRef[]
  zones: ZoneDefinition[]
  resources: ResourceDefinition[]
  phases: PhaseMachine
  objectives: Objective[]
  commandDefinitions: CommandDefinition[]
  presentationTheme: PresentationThemeRef
  aiProfiles: AgentProfile[]
}
```

17.2 CommandDefinition

```
CommandDefinition {
  id: "play_card"
  availableDuring: PhaseId[]
  sourceSelector: Selector
  targetSelectors: Selector[]
  legality: PredicateScript[]
  costs: CostExpression[]
  effects: EffectScript[]
  ui: CommandUiDefinition
}
```

17.3 Selector

```
Selector {
  id: "target_enemy"
  type: Object | Slot | Zone | Player | Value | Mode
  count: number | Range
  zones?: ZoneRef[]
  predicate?: PredicateScript
  visibilityRequired?: Visibility
  ordering?: PlayerChoice | Random | Highest | Lowest | TopN | BottomN
}
```

选择器类型	用途
Self	引用自身。
Owner / Controller	所有者或操控者。
Selected Object	玩家指定对象。
Selected Slot	玩家指定格子。
Random Object	随机目标。
All Matching	全体符合者。
Top N / Bottom N	牌堆顶或牌堆底。
Adjacent Objects	相邻对象。
Opposing Slot Object	对位对象。
Highest / Lowest	最高攻击、最低血量等。

17.4 PresentationProfile

```
PresentationProfile {
  id: string
  theme: Casino | Arcane | Horror | Heroic | Mechanical
  defaultMove: MovePreset
  defaultDestroy: DestroyPreset
  cardMaterialSet: MaterialSetRef
  beatOverrides: Map<EventType, BeatTimelineRef>
  audioSet: AudioSetRef
  cameraRules: CameraRule[]
  fallback: PresentationProfileRef
}
```

18. 3D 资产与表现绑定规范

为了实现演出级卡牌，资产命名和绑定必须规范化。卡牌 Actor 不应只是一张 2D 图片，而应有网格、材质层、插槽、状态特效和动画状态机。

18.1 资产层级

资产	要求
CardMesh	支持厚度、边缘、弯曲、轻微形变；LOD 可配置。
FrontMaterial	卡面图、动态文字、稀有度边框、闪箔。
BackMaterial	牌背、阵营、规则集主题。
FoilLayer	稀有牌光泽、动态反射。
DecalLayer	印记、诅咒、血迹、裂痕、升级标记。
Sockets	VFX 挂点，如中心、四角、边缘、目标箭头起点。
AnimationClips	flip、hover、draw、play、slam、burn、shatter、slot_enter。
AudioEvents	纸张、筹码、火焰、刻印、怪物低语、法术释放。

18.2 ZoneAnchor

```
ZoneAnchor {
  zoneId: ZoneId
  anchorType: DeckPile | HandFan | DiscardPile | BoardSlot | MarketSlot | RewardSlot
  transformProvider: LayoutFunction
  capacityVisualPolicy: Stack | Fan | Grid | Scroll | Hidden
  cameraFocusPoint: Transform
}
```

18.3 UI 与 3D 同步

- 手牌、牌堆、弃牌堆等关键区域应既有规则 Zone，也有表现 Anchor。
- UI 数字变化应绑定 ResourceChanged，而不是手动同步。
- 可指定目标高亮应由合法性系统输出候选集，UI 只负责展示。
- 预览伤害、点数和胜率必须标记为预测值，不能写入 GameState。
- 动画跳过时，应直接把 CardActor 对齐到规则状态对应的 Anchor。

19. 存档、回放、随机与网络同步

19.1 Event Sourcing

推荐使用事件溯源：保存初始种子、规则集版本、内容版本、命令日志和必要快照。规则事件可由命令重放生成，或者作为调试冗余保存。

```
SavePackage {
  profileVersion: string
  rulesetVersion: string
  contentHash: string
  seed: string
  runSnapshot: RunStateSnapshot
  gameSnapshot?: GameStateSnapshot
  commandLog: Command[]
  eventLogChecksum: string
}
```

19.2 RNG 流

RNG 流	用途
rng.map	地图生成、节点分布。
rng.reward	奖励草案、稀有度、商店。
rng.combat	抽牌、随机目标、战斗规则随机。
rng.enemy	敌人行动选择。
rng.narrative	事件选项和剧情变体。
rng.presentation	粒子、镜头抖动、音效变体。
rng.cosmetic	非规则皮肤、装饰。

硬性规则：规则 RNG 与表现 RNG 必须分离。抽到哪张牌不能因为粒子随机数多生成了一个火星而改变。

19.3 同步策略

若未来支持联机或异步对战，优先采用 lockstep command replication。客户端只发送 Command，不发送结果。双方基于相同种子、规则集版本和内容 hash 独立模拟，并通过 event checksum 检测分歧。

20. 自动化测试、平衡与遥测

20.1 规则单元测试

```
Given:
  player.energy = 3
  player.hand contains card.strike
  enemy.hp = 20
When:
  PlayCard(card.strike, target=enemy)
Then:
  player.energy == 2
  enemy.hp == 14
  card.strike.zone == player.discard
  events contains CardPlayed and DamageDealt
```

20.2 Golden Replay

每个核心规则集维护固定种子回放。修改规则、效果、触发排序或内容数据后自动重放，发现结果变化时输出第一个差异事件。

```
seed_001: starter_combat_run
seed_002: blackjack_high_risk_cheat_run
seed_003: sacrifice_board_boss_run
seed_004: reward_and_shop_economy_run

Diff report:
  Event 134 changed
  Expected: DamageDealt amount=6
  Actual:   DamageDealt amount=8
  Cause:    modifier layer changed
```

20.3 平衡指标

指标	意义
胜率	总体难度。
平均战斗回合数	节奏与冗长程度。
平均每回合伤害	输出曲线。
平均抽牌数	过牌强度。
卡牌选择率	奖励吸引力。
卡牌打出率	实际可用性。
遗物胜率提升	遗物强度。
经济膨胀率	金币与商店平衡。
死亡节点分布	地图与难度曲线。
作弊成功/失败率	21 点风险机制平衡。
献祭收益率	献祭资源机制平衡。

20.4 Debug 工具

- 事件日志面板：按时间查看 Event、source、cause、payload。
- 触发队列面板：查看待结算触发、优先级和来源。
- 持续效果层面板：查看最终数值如何被每一层修改。

- RNG 面板：显示每个 RNG 流的调用记录和结果。
- 状态快照比较器：比较两次运行的 GameState 差异。
- 演出意图面板：查看哪些规则事件生成了哪些表现 Beat。

21. 编辑器与内容生产工具

工具	功能
Card Editor	编辑费用、目标、效果、关键词、升级、卡面、演出配置。
Relic / Artifact Editor	编辑触发、持续效果、稀有度、奖励池。
Mechanic Editor	注册资源、关键词、触发时机、标准效果和 UI 面板。
Encounter Editor	配置敌人、初始状态、特殊规则、奖励、Boss 阶段。
Run Map Editor	配置地图生成、节点池、Act 结构、难度曲线。
Reward Pool Editor	配置卡池、遗物池、权重、稀有度、解锁条件。
Presentation Timeline Editor	配置卡牌移动、摧毁、召唤、攻击、Boss 变身演出。
Rule Debugger	查看事件栈、触发队列、持续效果层、合法性失败原因。
Replay Viewer	回放任意一局，每一步暂停、快进、跳转。
Balance Dashboard	自动模拟后的胜率、卡牌强度和经济指标分析。

21.1 内容发布管线

Authoring Data
-> Schema Validation
-> Cross Reference Validation
-> Balance Static Checks
-> Asset Binding Validation
-> Localization Export
-> Content Hash Generation
-> Build Package
-> Runtime Load

21.2 策划防呆

- 卡牌效果必须能在编辑器中预览事件序列。
- 目标选择器必须显示示例合法目标。
- 费用表达式必须显示最大、最小和不可支付场景。
- 触发必须显示监听事件、优先级、次数限制和可能循环风险。
- 奖励池必须显示稀有度权重、解锁条件和冲突标签。
- 表现配置缺失时必须显示 fallback，而不是运行时空引用。

22. 开发路线图

阶段	目标	交付物
阶段一：确定性规则内核	完成对象、区域、事件、效果、触发、目标、RNG。	日志式可运行规则 demo；无 3D 也可验证抽牌、打牌、伤害、胜负。
阶段二：回合战斗纵切	实现爬塔式战斗最小闭环。	HP、Energy、Block、抽弃牌、敌人意图、三选一卡奖励。
阶段三：3D 卡牌桌面纵切	实现核心卡牌演出。	手牌扇形、抽牌飞入、打牌飞向目标、弃牌入堆、销毁燃烧。
阶段四：21 点规则集	验证非战斗牌类规则。	Shoe、Hit、Stand、Bust、DealerPolicy、Bet、Payout、作弊卡。

阶段五：位置制献祭规则集	验证棋盘、献祭和路线战斗。	LaneBoard、Slot、Creature、Sacrifice、LaneAttack、ScaleObjective。
阶段六：编辑器与模拟	进入可量产内容状态。	卡牌编辑器、遭遇编辑器、演出时间轴、回放、自动模拟、平衡面板。

22.1 MVP 定义

- 至少一个规则集可完整完成一局：新局、地图、遭遇、奖励、失败/胜利、结算。
- 至少 20 张卡、5 个遗物、5 种敌人、3 类节点、1 个 Boss。
- 核心动作都有 3D 表现：抽牌、打出、移动、弃牌、销毁、伤害、奖励选择。
- 同一 seed 可复现同一局结果。
- 编辑器能创建一张新卡并在不改代码的情况下进入运行时。
- 自动测试覆盖常规卡牌、触发、销毁、胜负、奖励和存档读取。

23. 验收标准

23.1 规则验收

项	标准
确定性	相同 seed、相同内容 hash、相同 CommandLog 产生相同 EventLog checksum。
扩展性	新增一种资源或关键词不需要修改 GameState 核心结构。
可配置目标	至少支持战斗击杀、21 点比较、天平阈值三种胜负模式。
触发系统	支持 Before、After、Replacement、Prevention、OnceLimit 和稳定排序。
状态检查	支持血量归零死亡、爆牌失败、区域容量、目标消失等检查。

23.2 表现验收

项	标准
卡牌移动	任意 Zone -> Zone 移动都能找到 Anchor 和 fallback 动画。
销毁演出	普通销毁和史诗销毁均可播放，跳过后对象状态正确。
镜头调度	多个连续事件不应产生过度跳镜；关键事件可升格为 cinematic。
UI 同步	资源、伤害、点数、目标预览均来自规则事件或预测接口。
性能	快速模式下大量抽弃牌不应阻塞；表现可降级。

23.3 内容工具验收

- 策划能用编辑器创建卡牌、遗物、敌人意图、奖励和简单事件。
- 错误配置能在构建前被发现，并定位到字段。
- 每张卡能预览规则事件序列和演出 profile。
- 奖励池能显示权重、稀有度和解锁条件。
- 回放工具能定位任意一条规则事件并显示导致它的 Command。

24. 风险与约束

风险	影响	缓解
----	----	----

规则与表现耦合	动画 bug 可能改变规则或卡死游戏。	严格 EventLog -> Intent -> Beat 单向管线；动画不可写 GameState。
触发循环	无限触发导致战斗无法结束。	触发深度限制、循环检测、onceLimit、调试报告。
内容爆炸	机制越多越难平衡。	插件上线清单、遥测指标、自动模拟、内容标签和池管理。
表格化配置难以表达复杂 Boss	过度 DSL 会变成另一门难维护语言。	声明式处理 80%内容，沙盒脚本处理特殊 Boss 和谜题。
3D 演出拖慢节奏	玩家反复游玩时感到冗长。	演出等级、快速模式、跳过、合并小事件。
确定性被破坏	回放、网络、测试失效。	规则 RNG 分流、内容 hash、CommandLog、checksum。

24.1 性能预算建议

对象	预算建议
规则 Tick	单次命令结算应以毫秒级完成；复杂 AI 可异步预估但提交命令必须同步验证。
可见 CardActor	手牌、场上、商店和奖励展示优先；不可见牌堆使用聚合 Actor。
VFX	普通打牌使用轻量粒子；Boss 和稀有事件使用高成本特效。
日志	运行时保留必要事件，长局可按节点压缩历史。
自动模拟	完全跳过表现层，直接使用规则内核。

25. 术语表与附录

25.1 专业术语表

术语	定义
Game State / 游戏状态	当前所有规则事实。
Object / 对象	可被规则引用的实体。
Definition / 定义	局外模板。
Instance / 实例	局内具体对象。
Zone / 区域	手牌、牌堆、场上、墓地等容器。
Owner / 所有者	对象的原始归属。
Controller / 操控者	当前控制该对象的一方。
Cost / 费用	执行动作前支付的资源或代价。
Effect / 效果	改变状态的原子操作。
Event / 事件	已发生或将发生的规则事实。
Trigger / 触发	监听事件并产生效果的能力。
Replacement Effect / 替代效果	把将要发生的事替换成另一件事。
Prevention Effect / 防止效果	阻止伤害、移动、死亡等。
Continuous Effect / 持续效果	一段时间内持续修正规则。
Layer / 层	持续效果的固定计算顺序。
Keyword / 关键词	可复用能力标签。
Counter / 指示物	毒、燃烧、充能等可计数状态。
Token / 衍生物	临时生成对象。
Attachment / 贴附物	装备、印记、诅咒、强化。
Stack / Queue / 堆叠或队列	等待结算的效果或触发。
Priority / 优先权	多方响应或触发排序。

Timing Window / 时机窗口	可响应、可替代或可触发的时间点。
State-Based Action / 状态检查动作	血量归零死亡、爆牌失败等自动检查。
Last Known Information / 最后已知信息	对象离场后仍用于结算的信息。
Run / 单局	从开始到胜败的肉鸽流程。
Encounter / 遭遇	一场战斗、赌局、谜题或事件。
Node / 节点	地图上的一个选择点。
Draft / 草案选择	从候选奖励中选择。
Pool / 池	卡池、遗物池、敌人池、事件池。
Rarity / 稀有度	普通、罕见、稀有等。
Modifier / 修正器	临时或永久改变规则的对象。
Intent / 意图	敌人预告行动。
Seed / 随机种子	确保随机可复现。
Telemetry / 遥测	用于平衡和行为分析的数据。

25.2 新机制设计模板

Mechanic Design Template:

1. Name / Fantasy / Player Promise
2. Rule Summary
3. New Resources / Zones / Keywords
4. Costs and Effects
5. Timing Windows and Trigger Behavior
6. UI Preview and Tutorial Requirements
7. Presentation Style and Required Assets
8. AI Evaluation Features
9. Balance Metrics
10. Edge Cases and Tests

25.3 新卡牌设计模板

Card Design Template:

```

id:
name:
rarity:
pools:
types:
cost:
timing:
targets:
effects:
upgrades:
keywords:
presentationProfile:
aiHint:
balanceTags:
tests:
  - legality
  - cost payment
  - expected events
  - trigger interactions
  - presentation fallback

```

25.4 结论

URCE 的关键不是把某个卡牌游戏做成模板，而是把卡牌桌游的经典规则抽象、肉鸽的生命周期、数据驱动内容生产和 3D 演出级表现拆成稳定、可测试、可扩展的层。只要坚持“规则事实”和“演出表现”分离，以对象、区域、事件、效果、触发和目标为基础，就可以在同一引擎中承载 21 点肉鸽、回合战斗爬塔、位置制献祭卡牌，以及后续原创混合玩法。